

Domain Driven Design

Building Microservice Systems

Sharif University of Technology, Spring 2026

Ali Abrishami

Seyyed Hossein Mirhosseini



Architectural Patterns



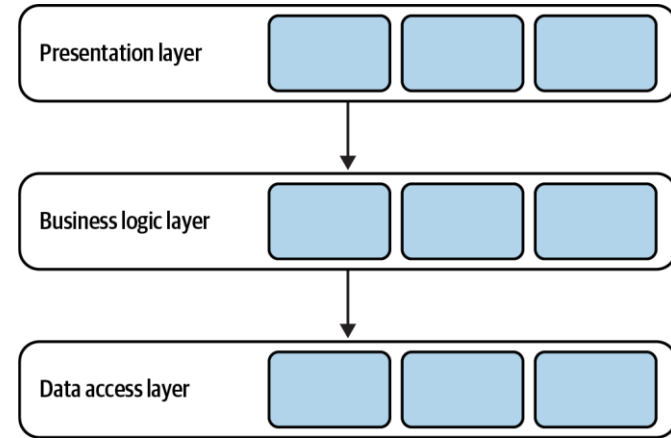
Architectural Patterns

- The tactical patterns are the different ways to model and implement business logic.
- Architectural patterns introduce **organizational principles** for the different aspects of a codebase and present clear boundaries between them.
- Let's explore three predominant application architecture patterns and their use cases:
 1. Layered Architecture
 2. Ports & Adapters
 3. CQRS.



Layered Architecture

- Layered architecture is one of the **most common** architectural patterns.
- It organizes the codebase into horizontal layers, with each layer addressing one of the following technical concerns:
 - Interaction with the consumers,
 - Implementing business logic,
 - And persisting the data.



Layered architecture



Presentation Layer

- The presentation layer implements the program's user interface for interactions with its consumers.
- In the pattern's original form, this layer denotes a **graphical** interface, such as a **web** interface or a **desktop** application.
- Strictly speaking, the presentation layer is the program's public interface.

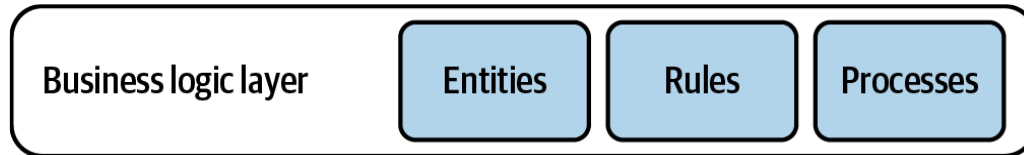


Presentation layer



Business Logic Layer

- This layer is responsible for implementing and encapsulating the program's business logic.
- This is the place where business decisions are implemented.
- As Eric Evans says: ***this layer is the heart of software.***
- This layer is where the business logic patterns are implemented.
 - For example, active records or a domain model.



Business logic layer



Data Access Layer

- The data access layer provides access to **persistence** mechanisms.
- Since the NoSQL revolution broke out, it is common for a system to work with multiple databases. For example:
 - A document store can act as the operational database,
 - A search index for dynamic queries,
 - And an **in-memory** database for **performance-optimized** operations.
- A cloud-based object storage or a message bus can be used to orchestrate communication between the different components.

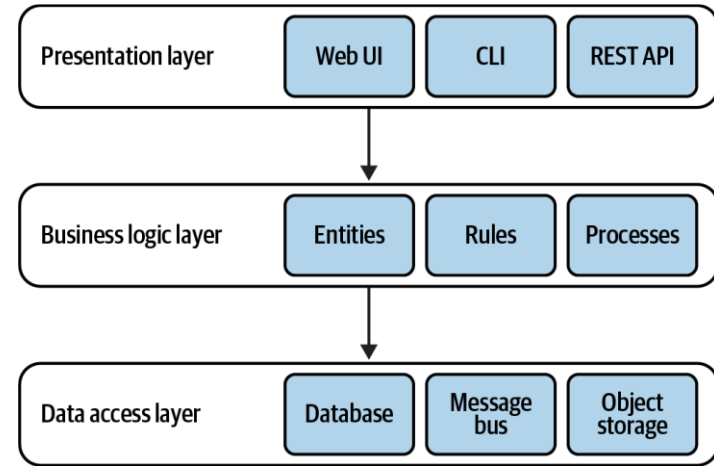


Data access layer



Communication Between Layers

- The layers are integrated in a **top-down** communication model:
Each layer can hold a dependency only on the layer **directly** beneath it.
- The presentation layer references only the business logic layer.
 - It has no knowledge of the design decisions made in the data access layer.



Layered architecture

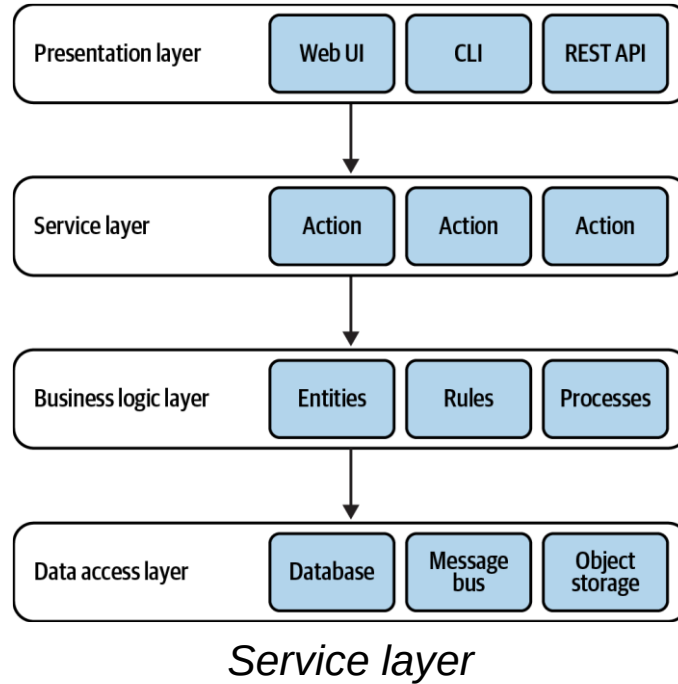


Variation

- It's common to see the layered architecture pattern extended with an additional layer: **the service layer**.
- *Service layer defines an application's boundary with a **layer of services** that establishes a set of available **operations** and **coordinates** the application's response in each operation.*
- The service layer acts as an **intermediary** between the program's presentation and business logic layers.



Service Layer



```

namespace ServiceLayer
{
    public class UserService
    {
        ...

        public OperationResult Create(ContactDetails contactDetails)
        {
            OperationResult result = null;

            try
            {
                _db.StartTransaction();

                var user = new User();
                user.SetContactDetails(contactDetails)
                user.Save();

                _db.Commit();
                result = OperationResult.Success;
            } catch (Exception ex) {
                _db.Rollback();
                result = OperationResult.Exception(ex);
            }

            return result;
        }
        ...
    }
}

```

```

namespace MvcApplication.Controllers
{
    public class UserController: Controller
    {
        ...

        [AcceptVerbs(HttpVerbs.Post)]
        public ActionResult Create(ContactDetails contactDetails)
        {
            var result = _userService.Create(contactDetails);
            return View(result);
        }
    }
}

```



Service Layer (cont.)

- It's important to note that in the context of the architectural pattern, the service layer is a logical boundary. It is **not** a physical service.
- The service layer acts as a **facade** for the business logic layer: it exposes an interface that corresponds with the public interface's methods, encapsulating the **required orchestration** of the underlying layers.



Service Layer (cont.)

- Having an explicit service level has a number of advantages:
 - We can reuse the same service layer to serve **multiple public interfaces**; for example, a graphical user interface and an API. No duplication of the orchestration logic is required.
 - It improves **modularity** by gathering all related methods in one place.
 - It further **decouples** the presentation and business logic layers.
 - It makes it easier to **test** the business functionality.



Terminology

- Elsewhere, you may encounter other terms used for the layered architecture:
 - Presentation layer = User interface layer
 - Service layer = Application layer
 - Business logic layer = Domain layer = Model layer
 - Data access layer = Infrastructure layer



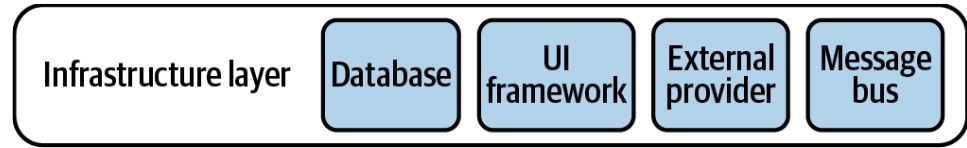
When to Use Layered Architecture

- This architectural pattern is a good fit for a system with its business logic implemented using the ***transaction script*** or ***active record*** patterns.
- However, the pattern makes it challenging to implement a ***domain model***.
- In a domain model, the business entities (aggregates and value objects) should have no dependency and no knowledge of the underlying infrastructure.
- The layered architecture's top-down dependency requires jumping through some hoops to fulfill this requirement.



Ports & Adapters

- The ports & adapters architecture addresses the **shortcomings** of the layered architecture and is a better fit for implementation of **more complex** business logic.
- Essentially, both the presentation layer and data access layer represent integration with **external components**: databases, external services, and user interface frameworks.
- These technical implementation details do not reflect the system's business logic; so, let's unify all such infrastructural concerns into a single **"infrastructure layer"**

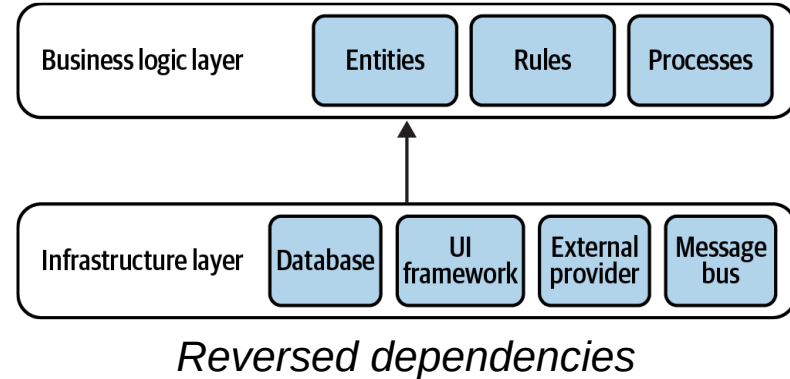


Presentation and data access layers combined into an infrastructure layer



Dependency Inversion Principle

- The dependency inversion principle (DIP) states that **high-level modules, which implement the business logic, should not depend on low-level modules.**
- However, that's precisely what happens in the traditional layered architecture. The business logic layer depends on the infrastructure layer.
- To conform with the DIP, let's reverse the relationship.

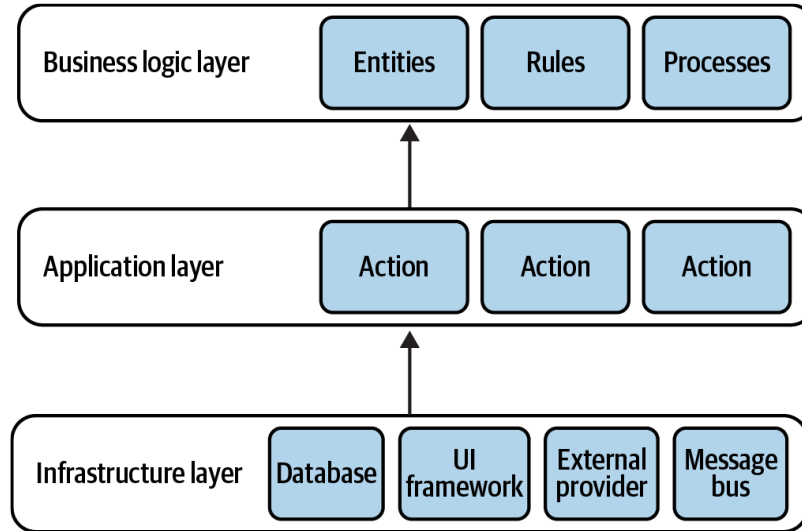


Application Layer

- Instead of being sandwiched between the technological concerns, now the **business logic** layer takes the **central role**.
- It **doesn't** depend on any of the system's infrastructural components.
- Finally, let's add an **application layer** as a facade for the system's public interface.
- As the service layer in the layered architecture, it describes all the operations exposed by the system and orchestrates the system's business logic for executing them.



Application Layer (cont.)



Traditional layers of the ports & adapters architecture

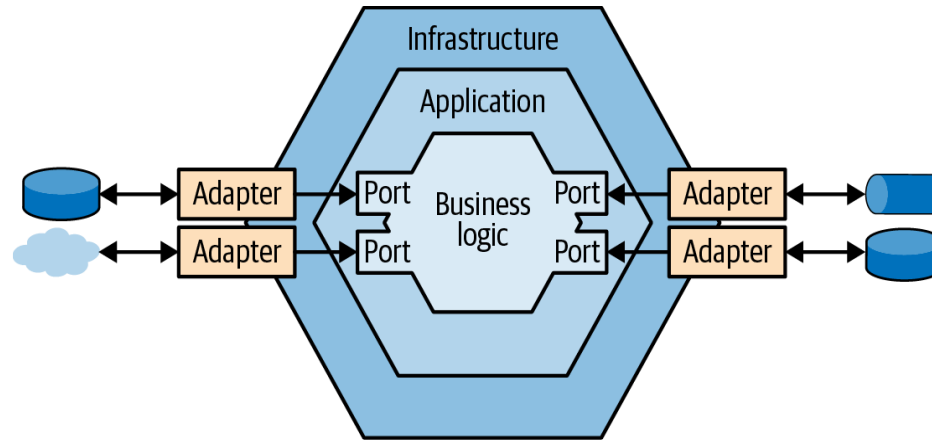


Integration of Infrastructural Components

- The core goal of the ports & adapters architecture is to **decouple** the system's business logic from its infrastructural components.
- Instead of referencing and calling the infrastructural components directly, the business logic layer defines “**ports**” that have to be implemented by the infrastructure layer.
- The infrastructure layer implements “**adapters**”: concrete implementations of the ports' interfaces for working with different technologies.



Ports & Adapters Architecture



Ports & adapters architecture



Other names

- The ports & adapters architecture is also known as **hexagonal** architecture, **onion** architecture, and **clean** architecture.
- All of these patterns are based on the same design principles, have the same components, and have the same relationships between them, but as in the case of the layered architecture, the terminology may differ:
 - Application layer = Service layer = Use case layer
 - Business logic layer = Domain layer = Core layer
- Ports & adapters architecture is a perfect fit for business logic implemented with the **domain model pattern**.



Command-Query Responsibility Segregation

- The command-query responsibility segregation (CQRS) pattern is based on the same organizational principles for business logic and infrastructural concerns as ports & adapters.
- It differs, however, in the way the **system's data** is managed.
- This pattern enables representation of the system's data in **multiple persistent models**.



Polyglot Modeling

- In many cases, it may be difficult, if not impossible, to use a single model of the system's business domain to address all of the system's needs.
- There is **no perfect database**.
- Or, as Greg Young says, all databases are flawed, each in its own way: *we often have to balance the needs for scale, consistency, or supported querying models.*
- An alternative to finding a perfect database is the polyglot persistence model: using **multiple databases** to implement different data-related requirements.



Polyglot Modeling (cont.)

- For example, a single system might use:
 - A **document store** as its operational database,
 - A **column store** for analytics/reporting,
 - A **search engine** for implementing robust search capabilities.
- The CQRS pattern provides the possibility of materializing projected models into **physical databases** that can be used for flexible querying options.



Implementation

- The pattern segregates the responsibilities of the system's models.
- There are two types of models:
 - The **command execution model**,
 - The **read models**.
- Command execution model:
 - CQRS devotes a single model to executing operations that **modify** the system's state (system **commands**).
 - This model is used to implement the business logic, validate rules, and enforce invariants.
 - The command execution model is also the only model representing strongly **consistent** data—the system's source of truth.



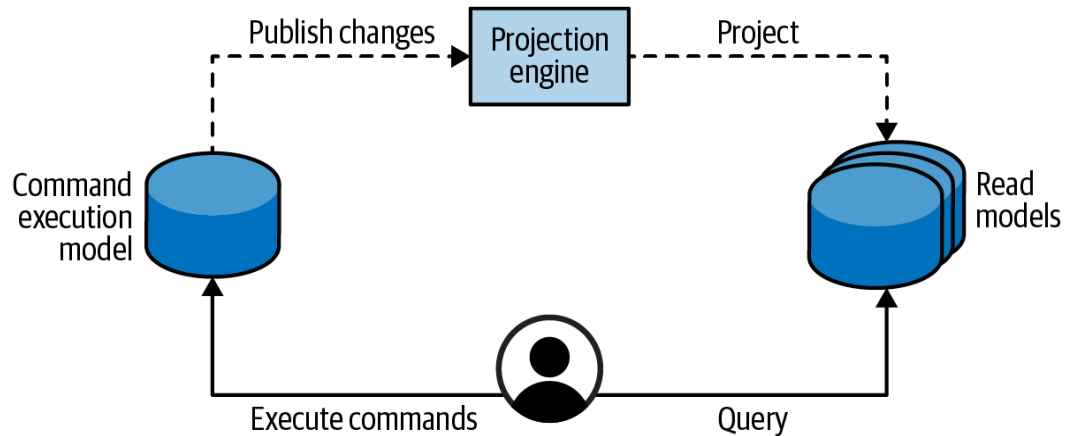
Implementation (cont.)

- Read models (projections):
 - The system can define as many models as needed to **present** data to users or **supply** information to other systems.
 - A read model is a precached projection. It can reside in a durable database, flat file, or in-memory cache.
 - None of the system's operations can directly modify the read models' data.



Implementation (cont.)

- For the read models to work, the system has to project changes from the command execution model to all its read models.

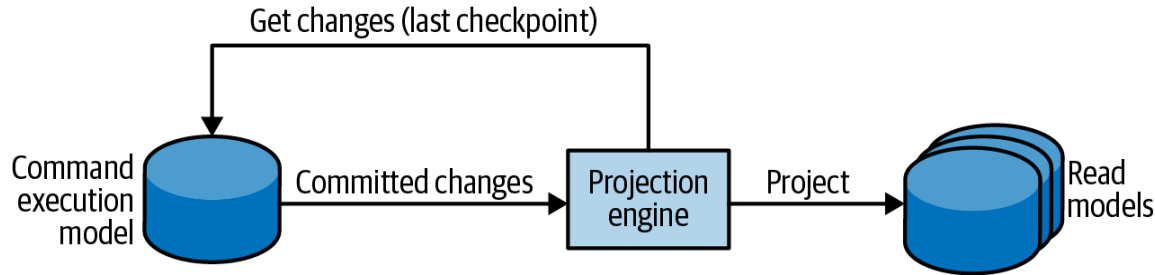


CQRS architecture



Synchronous projections

- Synchronous projections fetch changes to the OLTP data through the catch-up subscription model:
 - The projection engine **queries** the OLTP database for added or updated records.
 - The projection engine uses the updated data to regenerate/update the system's read models.

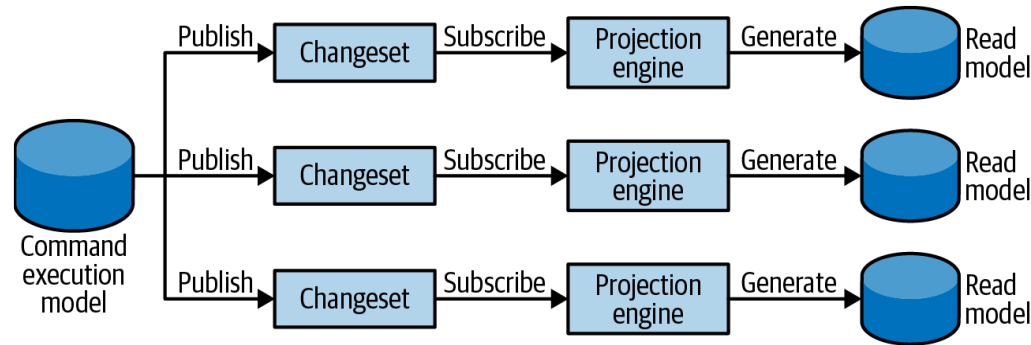


Synchronous projection model



Asynchronous projections

- The command execution model publishes all committed changes to a **message bus**.
- The system's projection engines can subscribe to the published messages and use them to update the read models.



Asynchronous projection of read models



Model Segregation

- In the CQRS architecture, the responsibilities of the system's models are **segregated** according to their type.
- A **command** can only operate on the strongly consistent command execution model.
- A **query** cannot directly modify any of the system's persisted state—neither the read models nor the command execution model.
- A command should always let the caller know whether it has succeeded or failed.
- *Therefore, a command can—and in many cases should—return data.*



When to Use CQRS

- The CQRS pattern can be useful for applications that need to work with the same data in multiple models, potentially stored in **different kinds of databases**.
- From an infrastructural perspective, CQRS allows for leveraging the strength of the different kinds of databases.
- For example, using a **relational database** to store the command execution model, a **search index** for full text search, and prerendered **flat files** for fast data retrieval, with all the storage mechanisms reliably synchronized.



Scope

- The patterns we've discussed—layered architecture, ports & adapters architecture, and CQRS—should **not** be treated as systemwide organizational principles.
- These are **not** necessarily high-level architecture patterns for a whole bounded context either.
- The subdomains can be of different types: core, supporting, or generic. Even subdomains of the same type may require different business logic and architectural patterns.
- Our goal is to drive design decisions according to the **actual needs** and **business strategy**.



Conclusion

- The **layered** architecture decomposes the codebase based on its technological concerns. Since this pattern couples business logic with data access implementation, it's a good fit for active record-based systems.
- The **ports & adapters** architecture inverts the relationships: it puts the business logic at the center and decouples it from all infrastructural dependencies. This pattern is a good fit for business logic implemented with the domain model pattern.
- The **CQRS** pattern represents the same data in multiple models. Although this pattern is obligatory for systems based on the event-sourced domain model, it can also be used in any systems that need a way of working with multiple persistent models.



References

- Learning Domain-Driven Design, *Vlad Khnonov*.
 - Chapter 8

